Day 106 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 2 reference

Watch: How to Prune Regression Trees, Clearly Explained!!! (StatQuest) | Cost Complexity Pruning (Theory + Code) (Meerkat Statistics)

`day106_pruning.py`, segment by segment, in the order `main()` uses things. Day 105's tree functions are imported unchanged, and scikit-learn supplies only the bundled dataset. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: R(T) = training errors of T / N · α = price per leaf · t = a node; T_t = the branch below it · g(t) = errors saved per extra leaf · β = geometric mean of two neighbouring alphas

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day105_decision_trees as d105
gini, grow, predict, size, show = d105.gini, d105.grow, d105.predict, d105.size, d105.show
stratified_split = d105.stratified_split
TITANIC = d105.TITANIC
FEATS = ["Pclass", "is_female", "Age", "SibSp", "Parch", "Fare"]
SPLITS = 30
NOISE = [0.0, 0.1, 0.2, 0.3, 0.4]
DEPTHS = [1, 2, 3, 4, 5, 6, 8, 10, None]
LEAVES = [1, 2, 5, 10, 20, 40, 80]
GAINS = [0.0, 0.002, 0.005, 0.01, 0.02, 0.05]
```

- `sys.path.insert(0, HERE)` — lets Python find day105_decision_trees.py next to this file.
- `with contextlib.redirect_stdout(io.StringIO()):` — anything printed during the import goes into a string, not the screen.
- `gini, grow, ... = d105.gini, d105.grow, ...` — shorter names for the same functions; nothing is copied or changed.

Segment 1 — Cost-complexity pruning

Small tree helpers: errors, copies, branches

CODE

```
def node_errors(node):
    return int(node["counts"].sum() - node["counts"].max())
def clone(node):
    c = dict(node)
    if "feature" in node:
        c["left"], c["right"] = clone(node["left"]), clone(node["right"])
    return c
def branch(node):
    if "feature" not in node:
        return node_errors(node), 1
    (ea, la), (eb, lb) = branch(node["left"]), branch(node["right"])
    return ea + eb, la + lb
def questions(node):
    if "feature" not in node:
        return []
    return [node] + questions(node["left"]) + questions(node["right"])
def cut(node):
    for k in ("feature", "threshold", "gain", "left", "right"):
        node.pop(k, None)
```

- `c = dict(node)` — a new dictionary with the same fields, so changes to the copy do not reach the original.
- `(ea, la), (eb, lb) = branch(...), branch(...)` — two recursive results, each a pair, unpacked at once.
- `[node] + questions(...) + questions(...)` — root first, then each side: the order stop_early relies on.
- `node.pop(k, None)` — removes a key; None stops an error if it is already gone.

cost_path: the weakest-link loop

MATH

$$g(t) = (R(t) – R(T_t)) / (|leaves(T_t)| – 1)$$

CODE

```
def cost_path(tree):
    n, t = int(tree["counts"].sum()), clone(tree)
    alphas, trees = [0.0], [clone(t)]
    while "feature" in t:
        nodes = questions(t)
        g = []
        for q in nodes:
            e, leaves = branch(q)
            g.append((node_errors(q) - e) / n / (leaves - 1))
        a = min(g)
        for q, v in zip(nodes, g):
            if v <= a + 1e-12:
                cut(q)
        alphas.append(a); trees.append(clone(t))
    return alphas, trees
```

- `while "feature" in t:` — continue while the root still asks a question.
- `g.append((node_errors(q) - e) / n / (leaves - 1))` — the formula above; leaves > 1 for every question, so it never divides by zero.
- `if v <= a + 1e-12:` — a small tolerance, so branches tied in floating point are cut together.
- `trees.append(clone(t))` — a snapshot: t keeps changing, so the list needs its own copy.

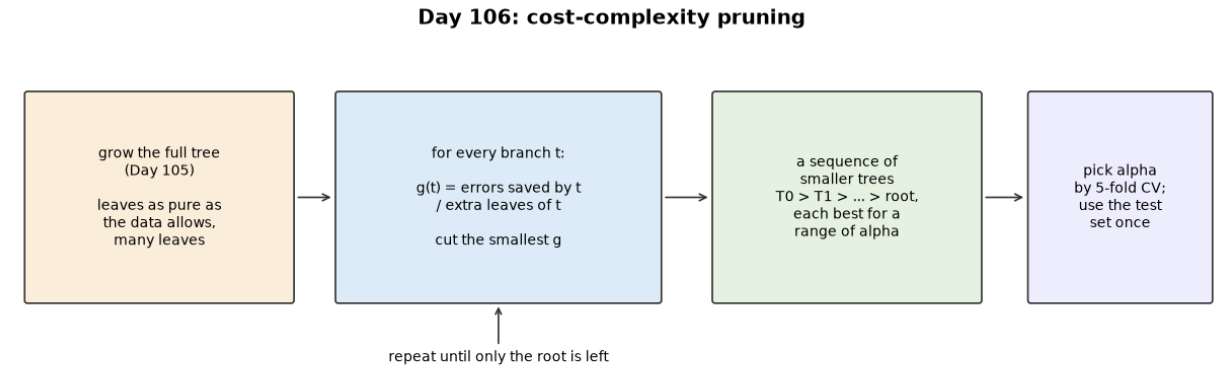
at_alpha and every_pruning

CODE

```
def at_alpha(alphas, trees, a):
    return trees[int(np.searchsorted(np.array(alphas), a + 1e-12, side="right")) - 1]
def cost(tree, a, n):
    e, leaves = branch(tree)
    return e / n + a * leaves
def every_pruning(node):
    leaf = {k: v for k, v in node.items() if k not in ("feature", "threshold", "gain", "left",
        "right")}
    if "feature" not in node:
        return [leaf]
    out = [leaf]
    for a in every_pruning(node["left"]):
        for b in every_pruning(node["right"]):
            out.append({**node, "left": a, "right": b})
    return out
```

- `np.searchsorted(a, v, side="right")` — where v would go in a sorted array, after any equal values; minus 1 gives the last alpha <= v.
- `{k: v for k, v in node.items() if k not in (...)}` — a dictionary comprehension: the node without its question.
- `{**node, "left": a, "right": b}` — a new dictionary: everything from node, with two fields replaced.

SYSTEM DIAGRAM



Grow, cut the weakest link, choose along the path.

Segment 2 — Choosing alpha

betas and cv_alpha

MATH

$$\beta_k = \sqrt{(\alpha_k \, \alpha_{k+1})}, \quad SE = std / \sqrt{k}$$

CODE

```
def betas(alphas):
    a = np.array(alphas)
    m = np.sqrt(np.maximum(a[:-1], 1e-12) * a[1:])
    return np.append(m, a[-1])
def cv_alpha(X, y, k=5, seed=0):
    n_classes = int(y.max()) + 1
    full = grow(X, y, n_classes, gini)
    alphas, trees = cost_path(full)
    b = betas(alphas)
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), k)
    scores = np.zeros((k, len(b)))
    for i in range(k):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(k) if j != i])
        al, ts = cost_path(grow(X[tr], y[tr], n_classes, gini))
        for j, a in enumerate(b):
            scores[i, j] = accuracy(at_alpha(al, ts, a), X[va], y[va])
    return alphas, trees, b, scores.mean(0), scores.std(0) / np.sqrt(k)
```

- `np.maximum(a[:-1], 1e-12)` — the first alpha is 0, which would make every geometric mean 0.
- `scores = np.zeros((k, len(b)))` — a table: one row per fold, one column per candidate.
- `scores.mean(0)` — axis 0 averages down the rows: one mean per candidate.

The one-SE choice, inside demo_cv

CODE

```
i = int(np.argmax(m))
ok = np.where(m >= m[i] - se[i])[0]
j = int(ok[np.argmax(b[ok])])
```

- `np.where(m >= m[i] - se[i])[0]` — the positions of every candidate within one standard error of the best.
- `ok[np.argmax(b[ok])]` — among those, the largest alpha, so the smallest tree.

Segment 3 — The experiments

cv_grid and compare

CODE

```
def cv_grid(X, y, make, grid, k=5, seed=0):
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), k)
    scores = []
    for g in grid:
        acc = []
        for i in range(k):
            va = folds[i]; tr = np.concatenate([folds[j] for j in range(k) if j != i])
            acc.append(accuracy(make(X[tr], y[tr], g), X[va], y[va]))
        scores.append(np.mean(acc))
    return grid[int(np.argmax(scores))]
def compare(X, y, seed):
    tr, te = stratified_split(y, 0.3, seed); out = {}
    full = grow(X[tr], y[tr], 2, gini)
    out["full tree"] = full
    alphas, trees, b, m, se = cv_alpha(X[tr], y[tr], seed=seed)
    i = int(np.argmax(m)); ok = np.where(m >= m[i] - se[i])[0]; j = int(ok[np.argmax(b[ok])])
    out["pruned, best CV"] = at_alpha(alphas, trees, b[i])
    out["pruned, one-SE"] = at_alpha(alphas, trees, b[j])
    d = cv_grid(X[tr], y[tr], lambda a, c, g: grow(a, c, 2, gini, max_depth=g), DEPTHS, seed=seed)
    out["depth limit"] = grow(X[tr], y[tr], 2, gini, max_depth=d)
    k = cv_grid(X[tr], y[tr], lambda a, c, g: grow(a, c, 2, gini, min_leaf=g), LEAVES, seed=seed)
    out["leaf-size limit"] = grow(X[tr], y[tr], 2, gini, min_leaf=k)
    return {n: (accuracy(t, X[te], y[te]), size(t)[0]) for n, t in out.items()}, d, k
```

- `make(X[tr], y[tr], g)` — make is a function passed as an argument, so one search serves every method.
- `lambda a, c, g: grow(a, c, 2, gini, max_depth=g)` — a small unnamed function: the depth-limit version of make.
- `{n: (...) for n, t in out.items()}` — every method's test accuracy and leaf count, under its name.

Label noise, inside demo_noise

CODE

```
tr, te = stratified_split(y, 0.3, s); rng = np.random.RandomState(100 + s)
yn = y.copy(); flip = rng.choice(tr, int(round(p * len(tr))), replace=False); yn[flip] = 1 -
    yn[flip]
```

- `rng.choice(tr, m, replace=False)` — m different training rows; test rows can never be chosen.
- `yn[flip] = 1 - yn[flip]` — swaps 0 and 1 at those positions in the noisy copy.

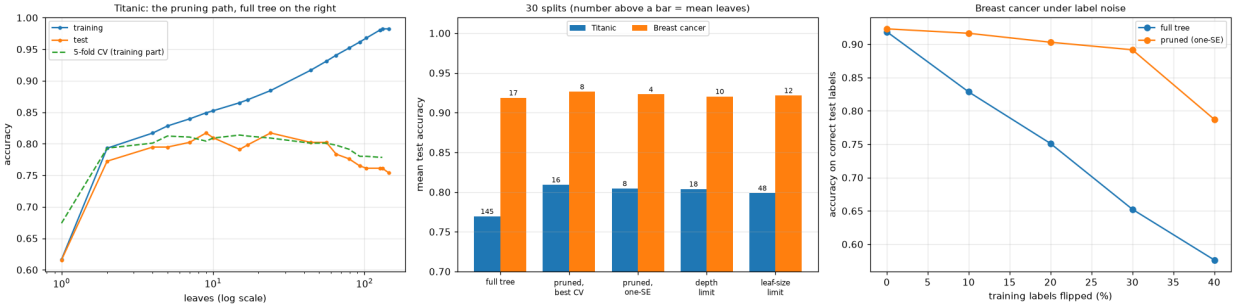
stop_early: pre-pruning by a minimum gain

CODE

```
def stop_early(node, min_gain):
    t = clone(node)
    for q in questions(t):
        if "feature" in q and q["gain"] < min_gain:
            cut(q)
    return t
```

- `for q in questions(t):` — root first: a cut removes everything below, which is why the next check is needed.
- `if "feature" in q and q["gain"] < min_gain:` — skip a node that an earlier cut already turned into a leaf.

REAL GRAPH



Real graph from the script: the Titanic pruning path, five methods over 30 splits, and label noise.

Quick Syntax Glossary — new or reinforced constructs

- `dict(node)` — a shallow copy of a dictionary.
- `{**a, "k": v}` — a new dictionary from a, with k set to v.
- `d.pop(k, None)` — remove a key if present.
- `np.searchsorted(a, v, side="right")` — the insertion position of v in sorted a.
- `np.unique(X, axis=0, return_inverse=True)` — the distinct rows, and which one each row is.
- `f(make, ...)` — passing a function as an argument.
- `lambda a, b: ...` — a small unnamed function.

End of Day 106 syntax reference. For the experiments and what they showed, see the Day 106 complete guide.